# TRM 可視化レイヤ — 4種の図式

対象: sakura-editor/sakura

生成: scripts/generate_visualization_pdf.py

## Sunburst

要求の種別階層を中心から放射状に配置。3秒で全体像把握。

## Sankey

種別×優先度の流量を横積みバーで表現。集中箇所と緊急度を可視化。

## Heatmap

state_variables × encapsulation_risks の交差表。複合リスクを定量表現。

## Chord

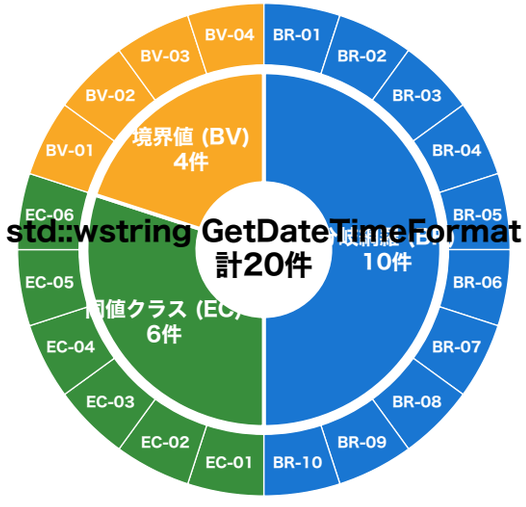
状態変数とリスクを円環配置、線種（実線/破線/点線）で重大度を強調。

対象ターゲット数: 8

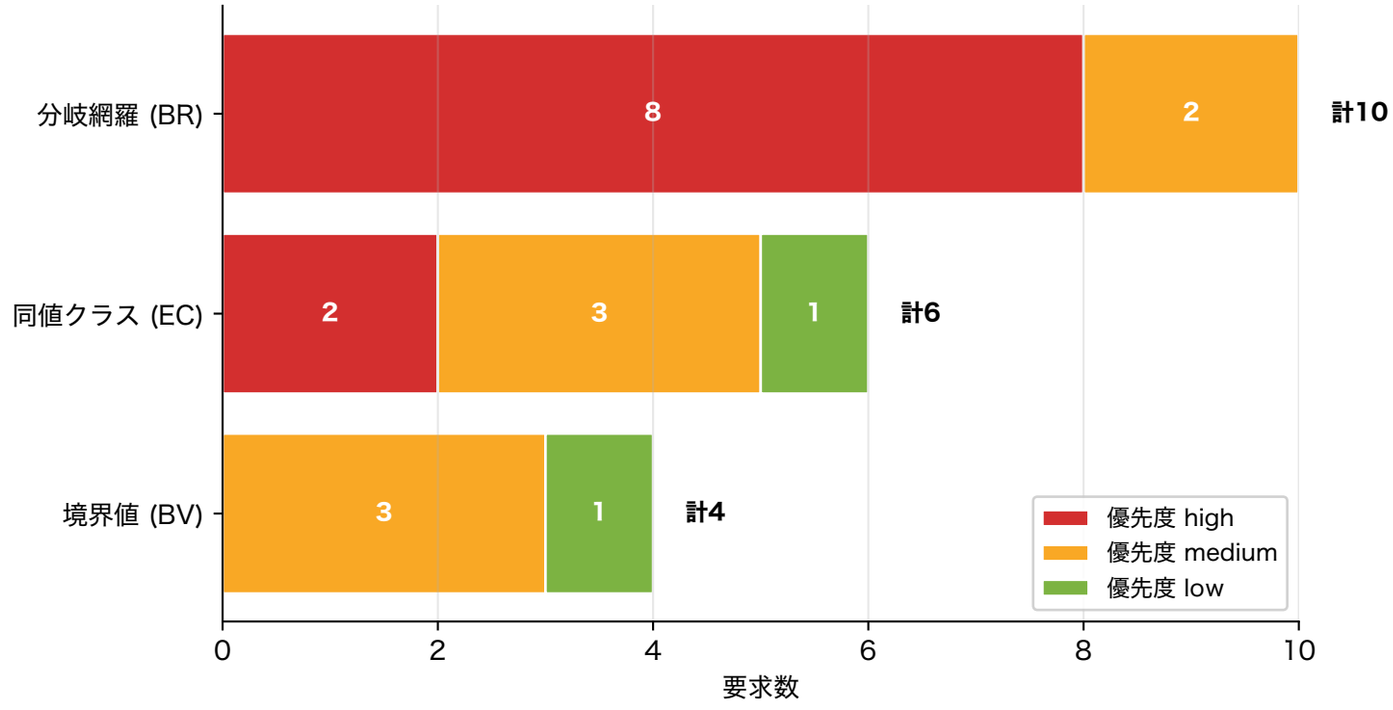
※ Heatmap/Chord は state_variables と encapsulation_risks を持つ
OOP対象（v3.1スキーマ）でのみ描画される。
手続き型・関数型対象では該当なしを明示表示する。

# 【4種の可視化レイヤ】sakura-editor/sakura / std::wstring GetDateTimeFormat

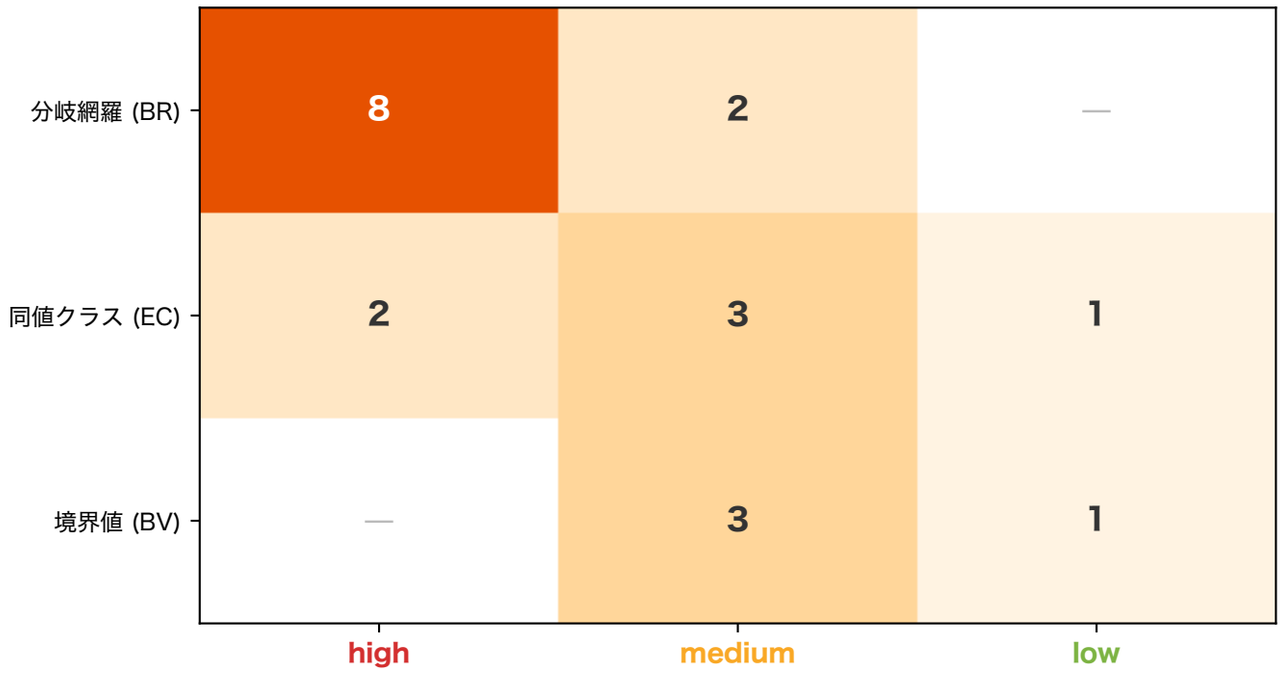
Sunburst: sakura-editor/sakura / std::wstring GetDateTimeFormat
— 要求の種別階層 —




Sankey: sakura-editor/sakura / std::wstring GetDateTimeFormat
— 種別×優先度の流量 —

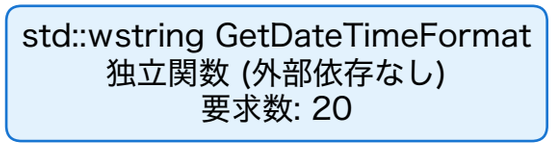



Heatmap: sakura-editor/sakura / std::wstring GetDateTimeFormat
— 要求種別 × 優先度 (手続き型代替表示) —




計: high=10, medium=8, low=2

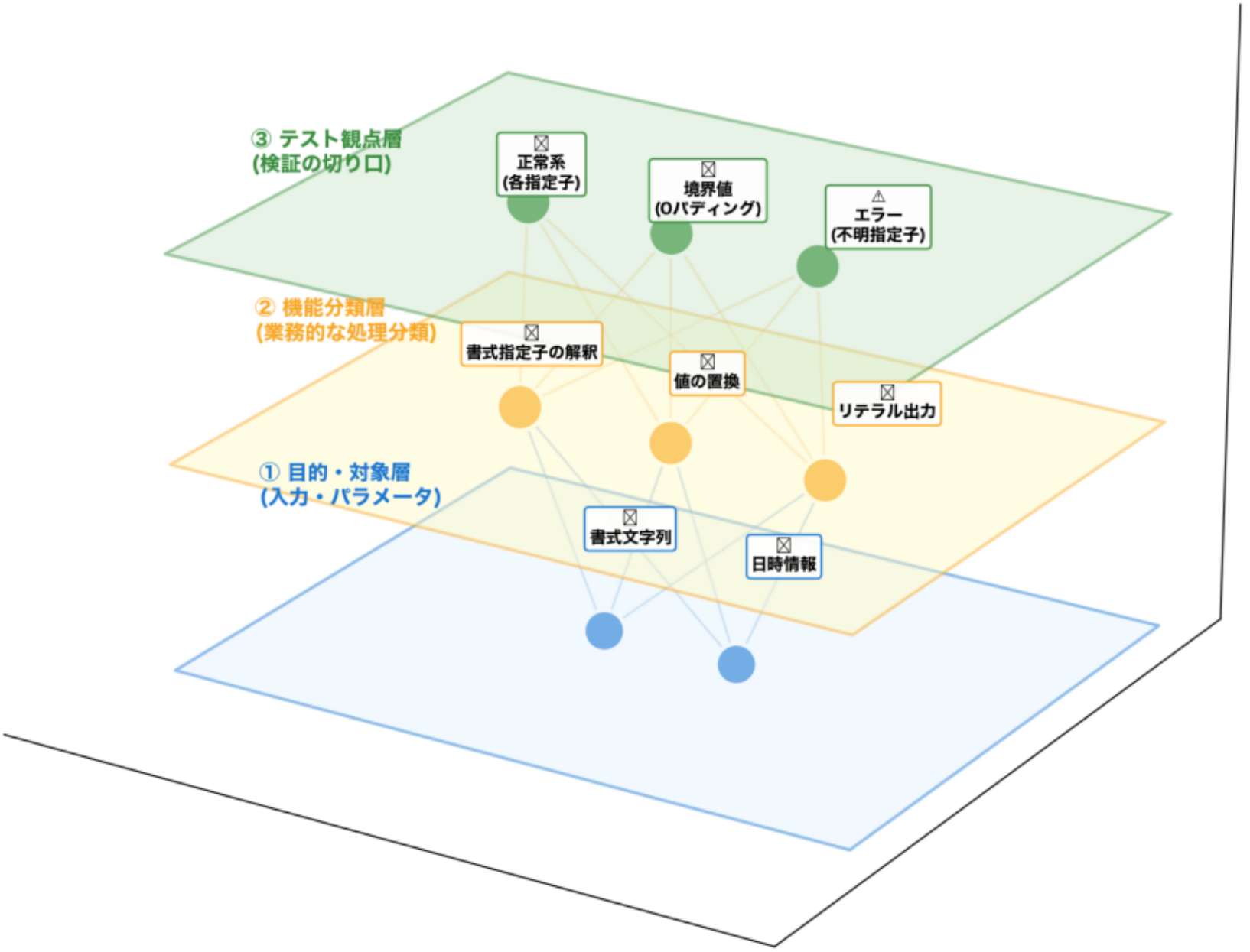
Chord: sakura-editor/sakura / std::wstring GetDateTimeFormat
— 依存関係 (手続き型代替) —



# 【機能分類ジオラマ】非エンジニア向け構造図

## 【sakura-editor】 std::wstring GetDateTimeFormat

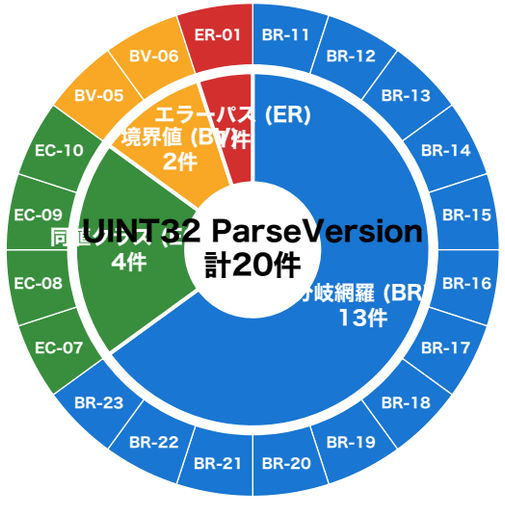
目的: 日時を指定書式で文字列化



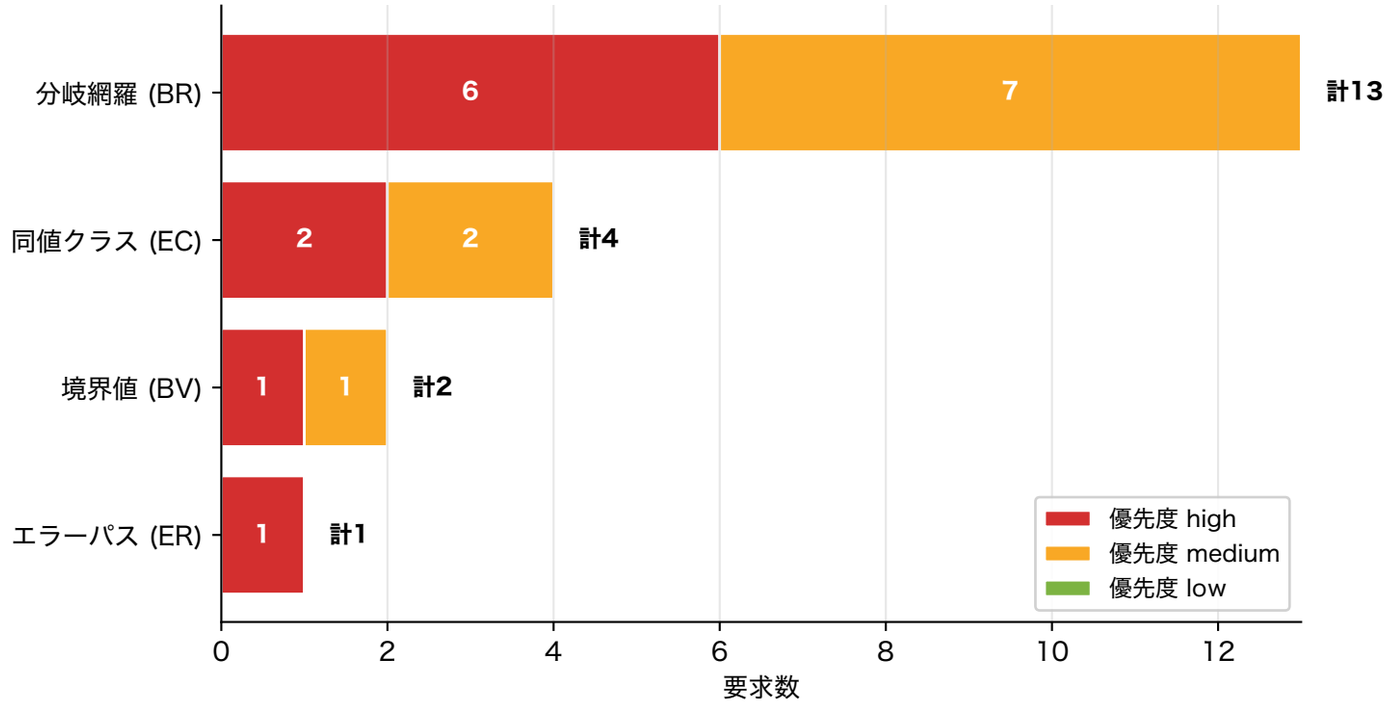

▼ 入力層 (何が入力か) → 機能分類層 (何を処理するか) → テスト観点層 (どう検証するか) ▼

# 【4種の可視化レイヤ】sakura-editor/sakura / UINT32 ParseVersion

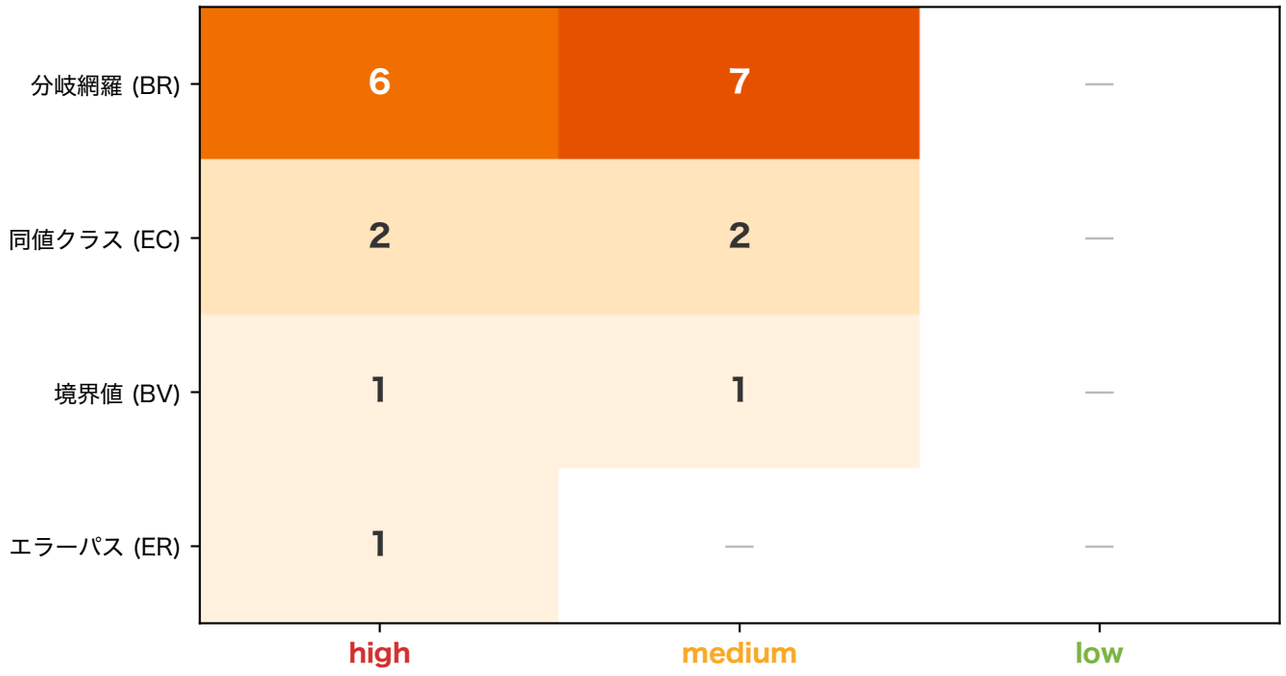
Sunburst: sakura-editor/sakura / UINT32 ParseVersion
— 要求の種別階層 —




Sankey: sakura-editor/sakura / UINT32 ParseVersion
— 種別×優先度の流量 —

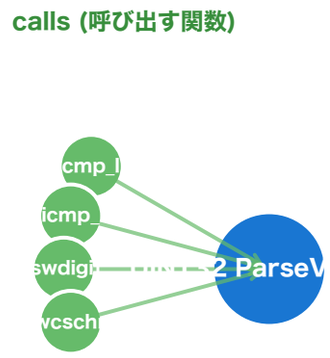



Heatmap: sakura-editor/sakura / UINT32 ParseVersion
— 要求種別 × 優先度 (手続き型代替表示) —




計: high=10, medium=10, low=0

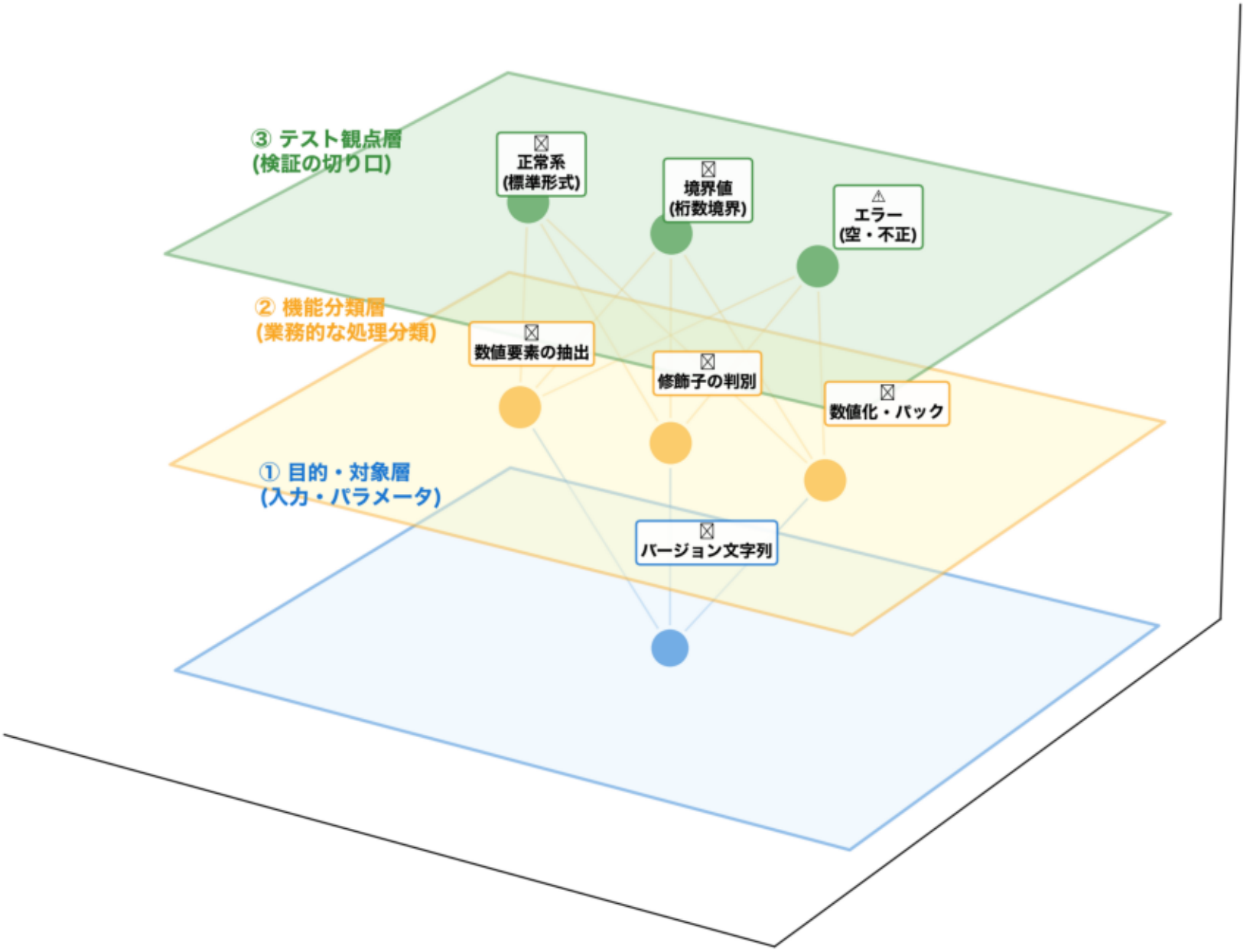
Chord: sakura-editor/sakura / UINT32 ParseVersion
— 依存関係マップ (手続き型代替) —



# 【機能分類ジオラマ】非エンジニア向け構造図

## 【sakura-editor】 UINT32 ParseVersion

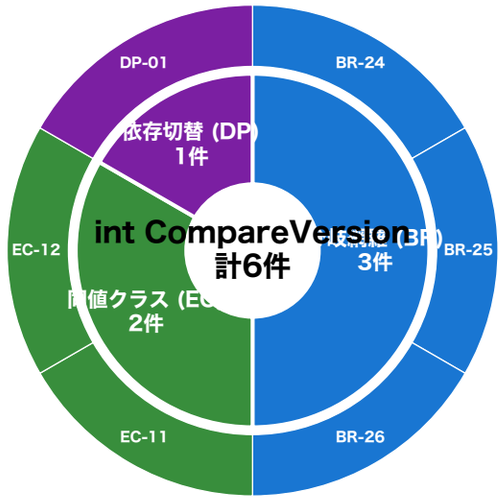
目的: バージョン文字列を数値にパース



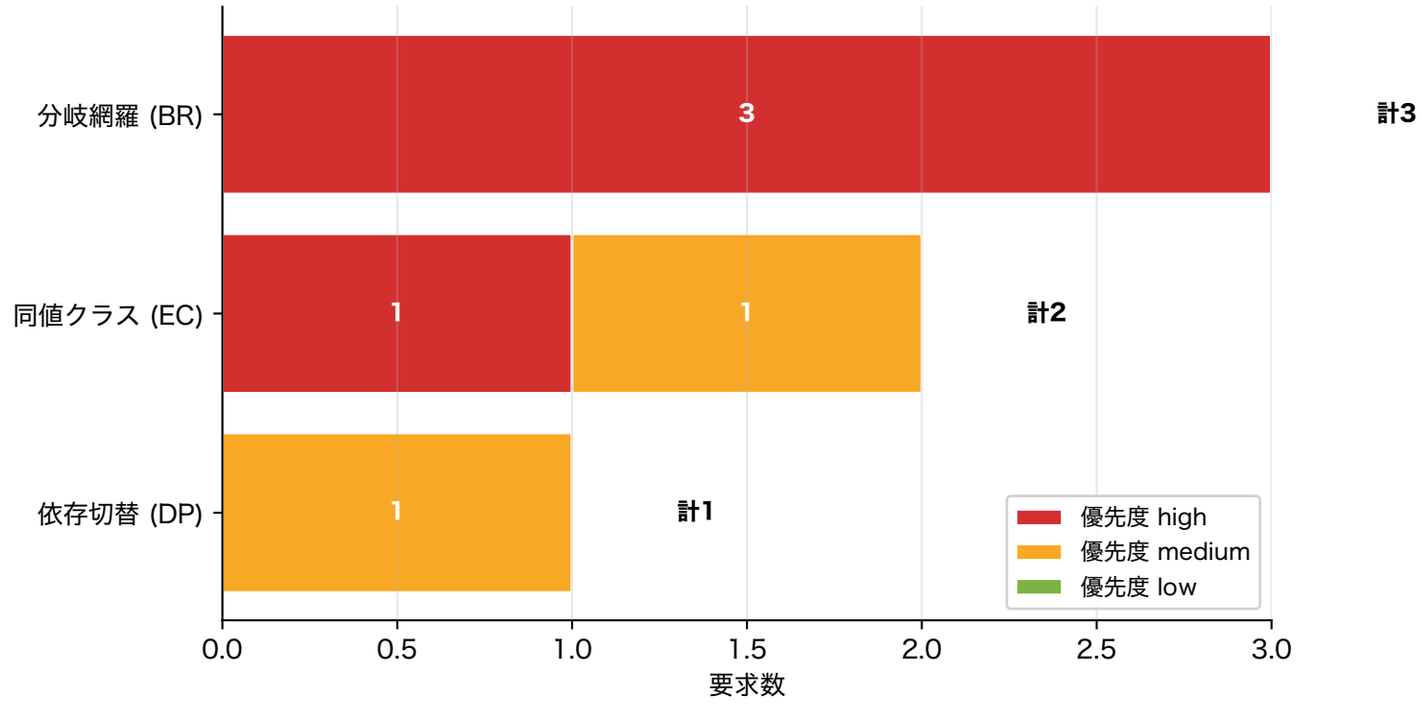

▼ 入力層 (何が入力か) → 機能分類層 (何を処理するか) → テスト観点層 (どう検証するか) ▼

# 【4種の可視化レイヤ】sakura-editor/sakura / int CompareVersion

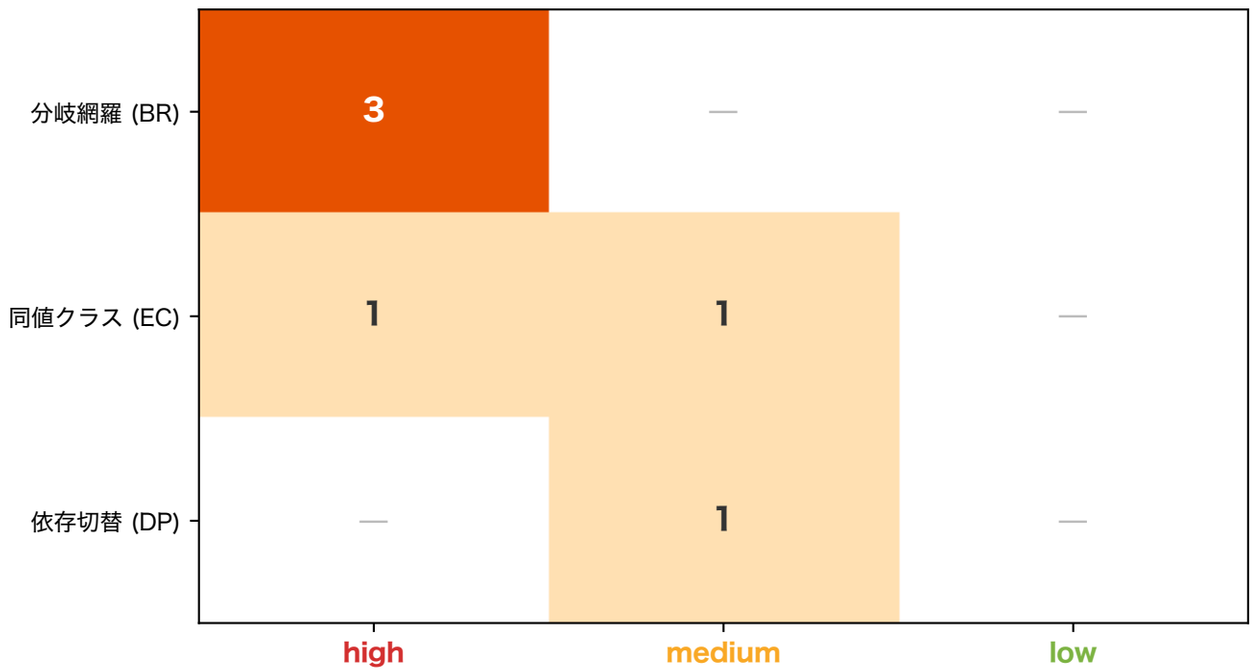
Sunburst: sakura-editor/sakura / int CompareVersion
— 要求の種別階層 —




Sankey: sakura-editor/sakura / int CompareVersion
— 種別×優先度の流量 —




Heatmap: sakura-editor/sakura / int CompareVersion
— 要求種別 × 優先度 (手続き型代替表示) —



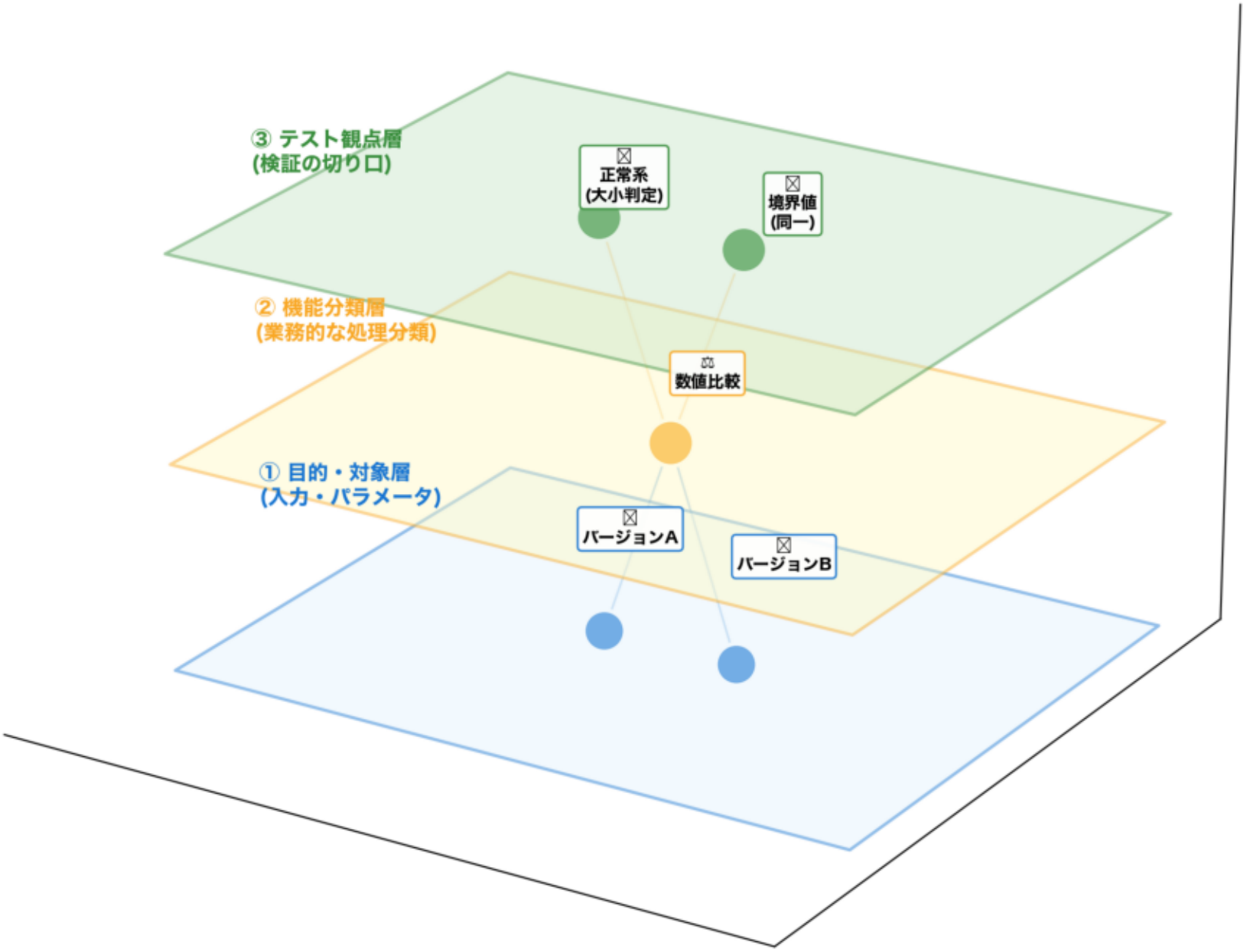

計: high=4, medium=2, low=0

Chord: sakura-editor/sakura / int CompareVersion
— 依存関係マップ (手続き型代替) —

# 【機能分類ジオラマ】非エンジニア向け構造図

## 【sakura-editor】 int CompareVersion

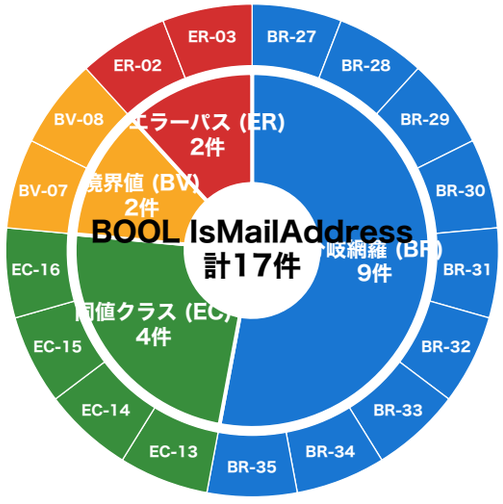
目的: 2つのバージョンを比較



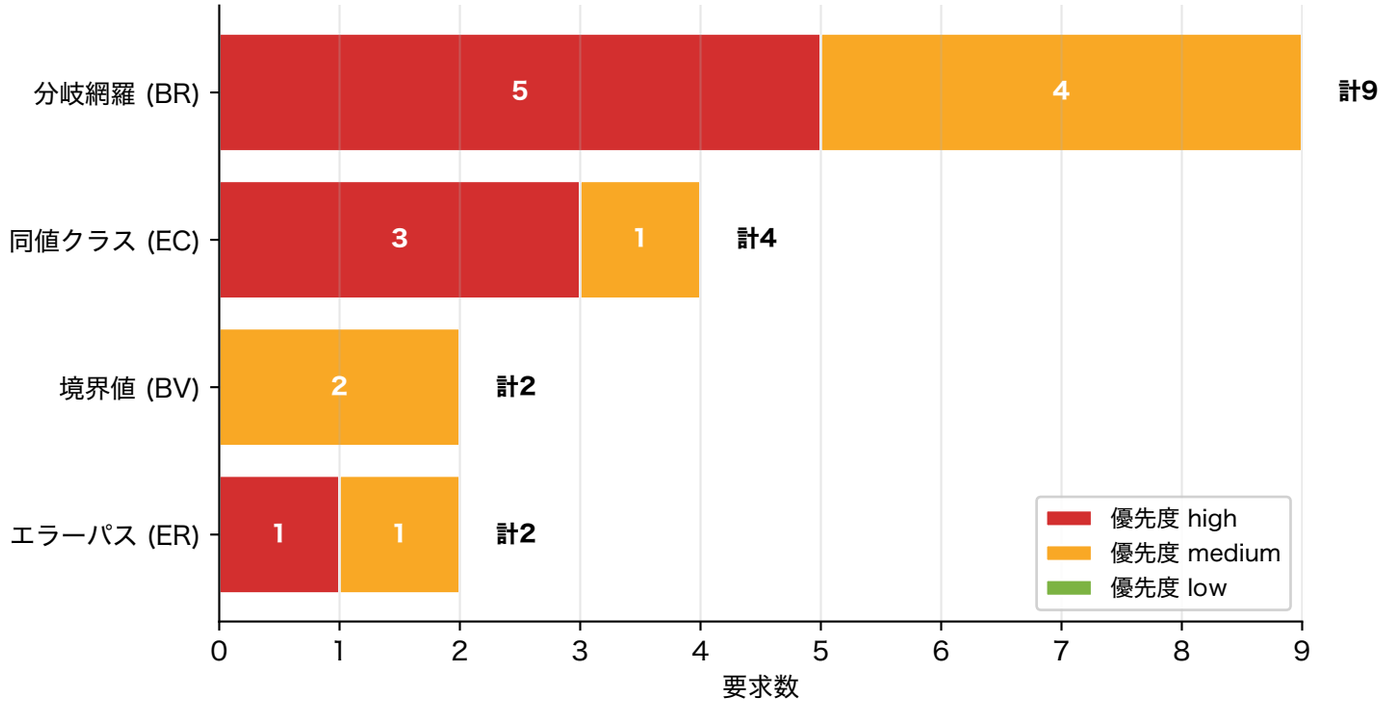

▼ 入力層 (何が入力か) → 機能分類層 (何を処理するか) → テスト観点層 (どう検証するか) ▼

# 【4種の可視化レイヤ】sakura-editor/sakura / BOOL IsMailAddress

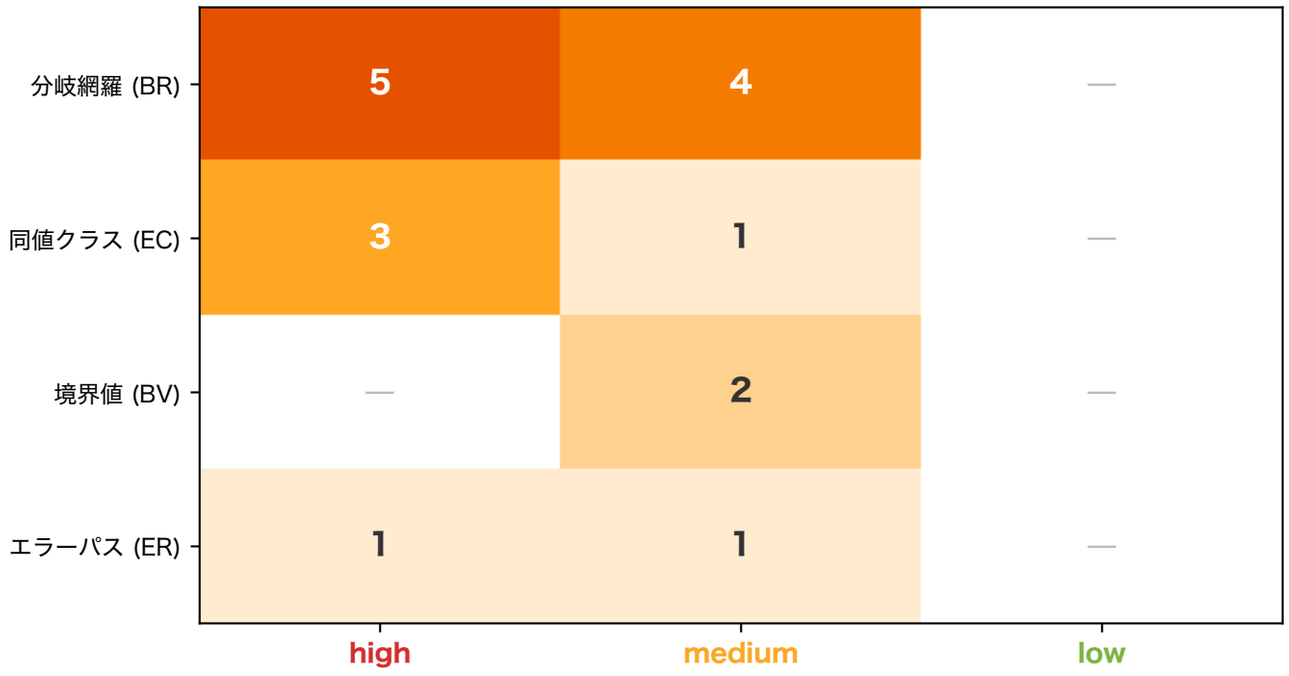
Sunburst: sakura-editor/sakura / BOOL IsMailAddress
— 要求の種別階層 —




Sankey: sakura-editor/sakura / BOOL IsMailAddress
— 種別×優先度の流量 —

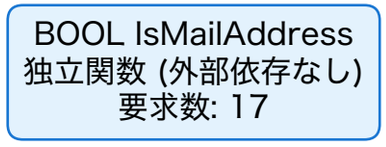



Heatmap: sakura-editor/sakura / BOOL IsMailAddress
— 要求種別 × 優先度 (手続き型代替表示) —




計: high=9, medium=8, low=0

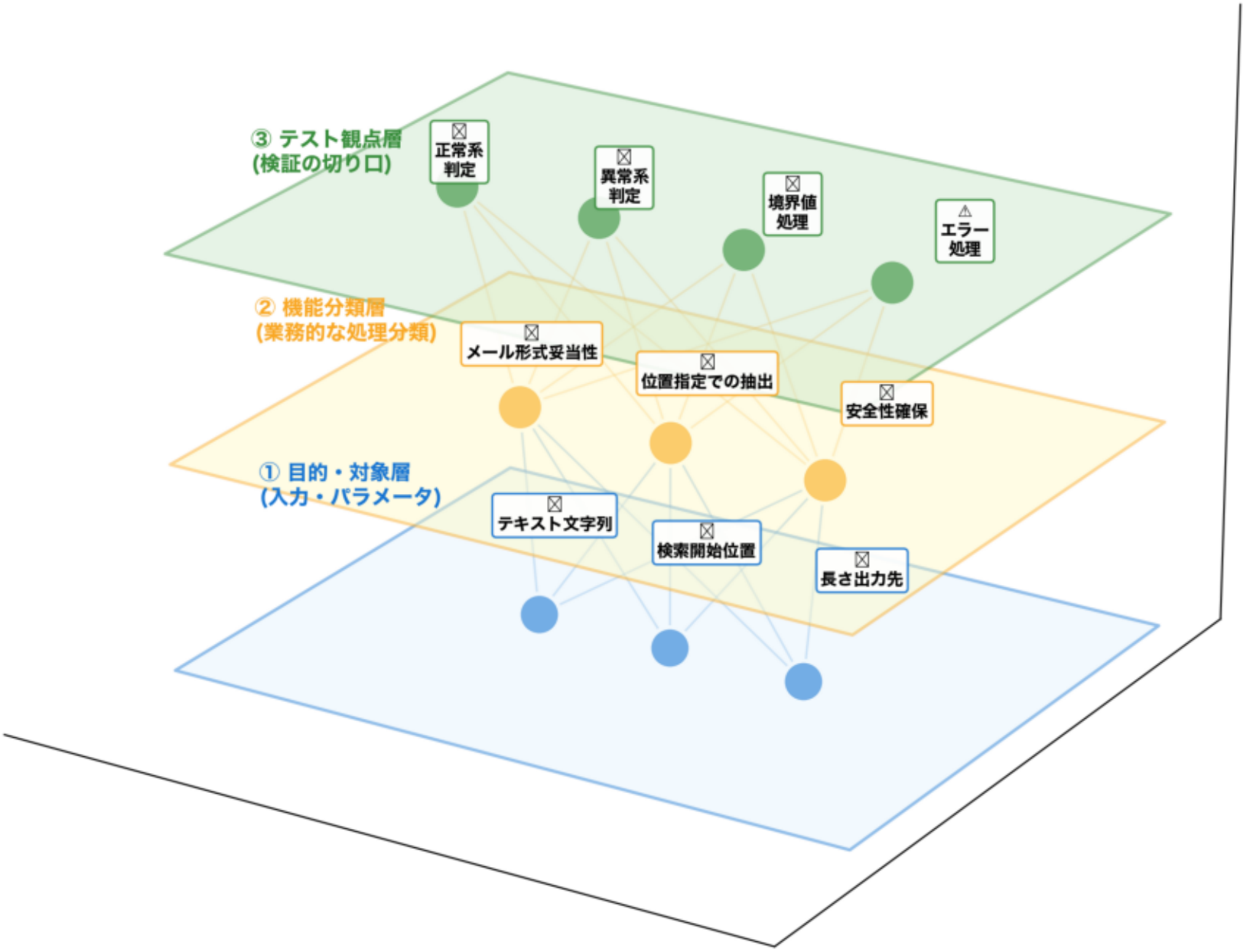
Chord: sakura-editor/sakura / BOOL IsMailAddress
— 依存関係 (手続き型代替) —



# 【機能分類ジオラマ】非エンジニア向け構造図

## 【sakura-editor】 BOOL IsMailAddress

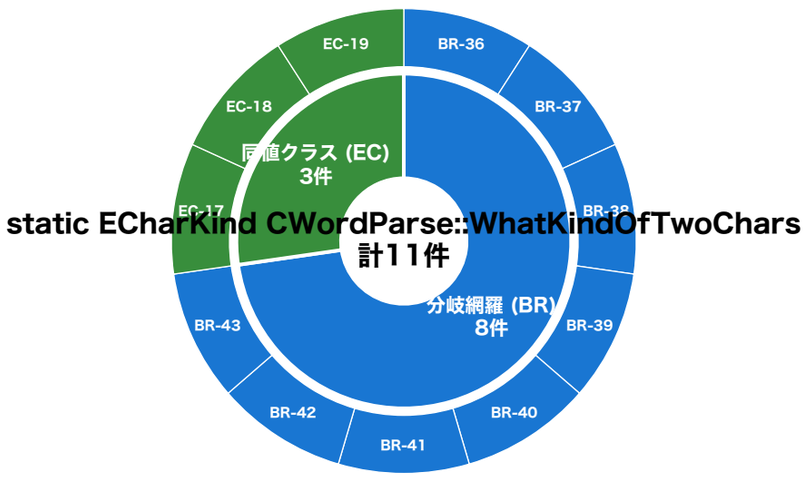
目的: 文字列がメールアドレス形式かを判定



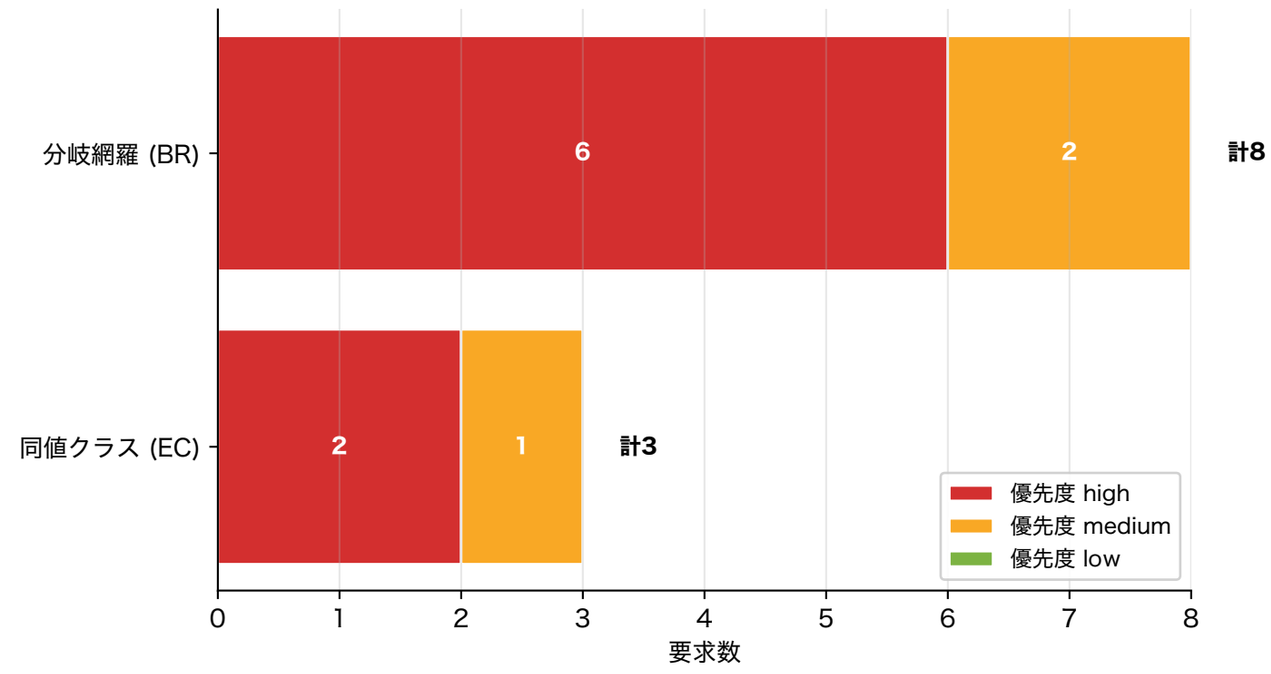

▼ 入力層 (何が入力か) → 機能分類層 (何を処理するか) → テスト観点層 (どう検証するか) ▼

# 【4種の可視化レイヤ】sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars

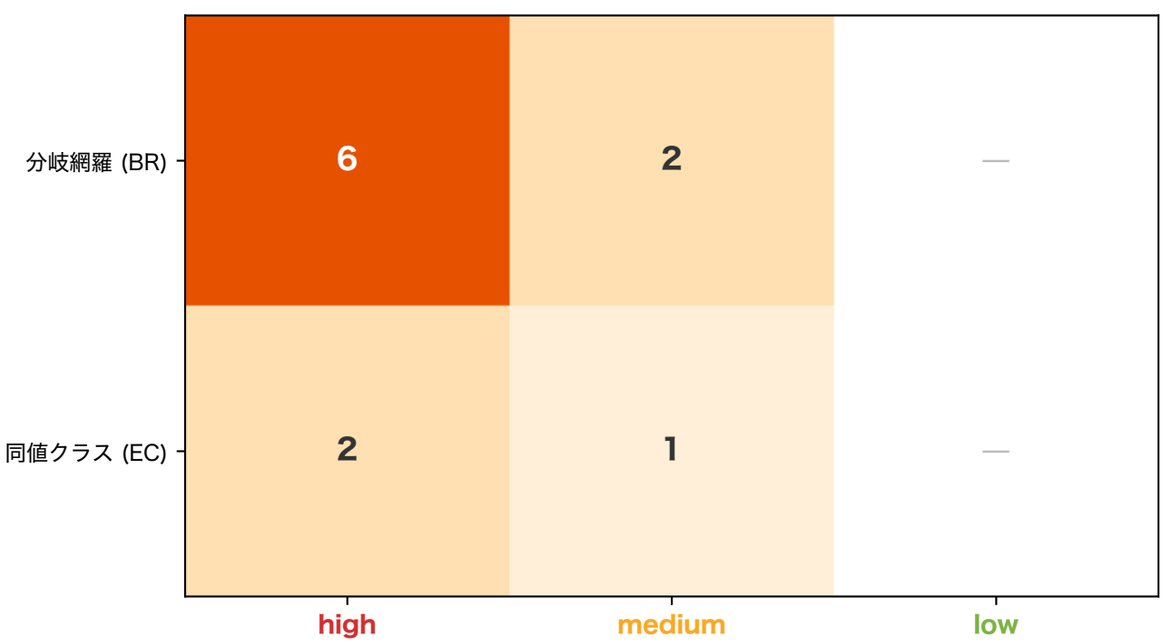
Sunburst: sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars
— 要求の種別階層 —




Sankey: sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars
— 種別×優先度の流量 —




Heatmap: sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars
— 要求種別 × 優先度 (手続き型代替表示) —




計: high=8, medium=3, low=0

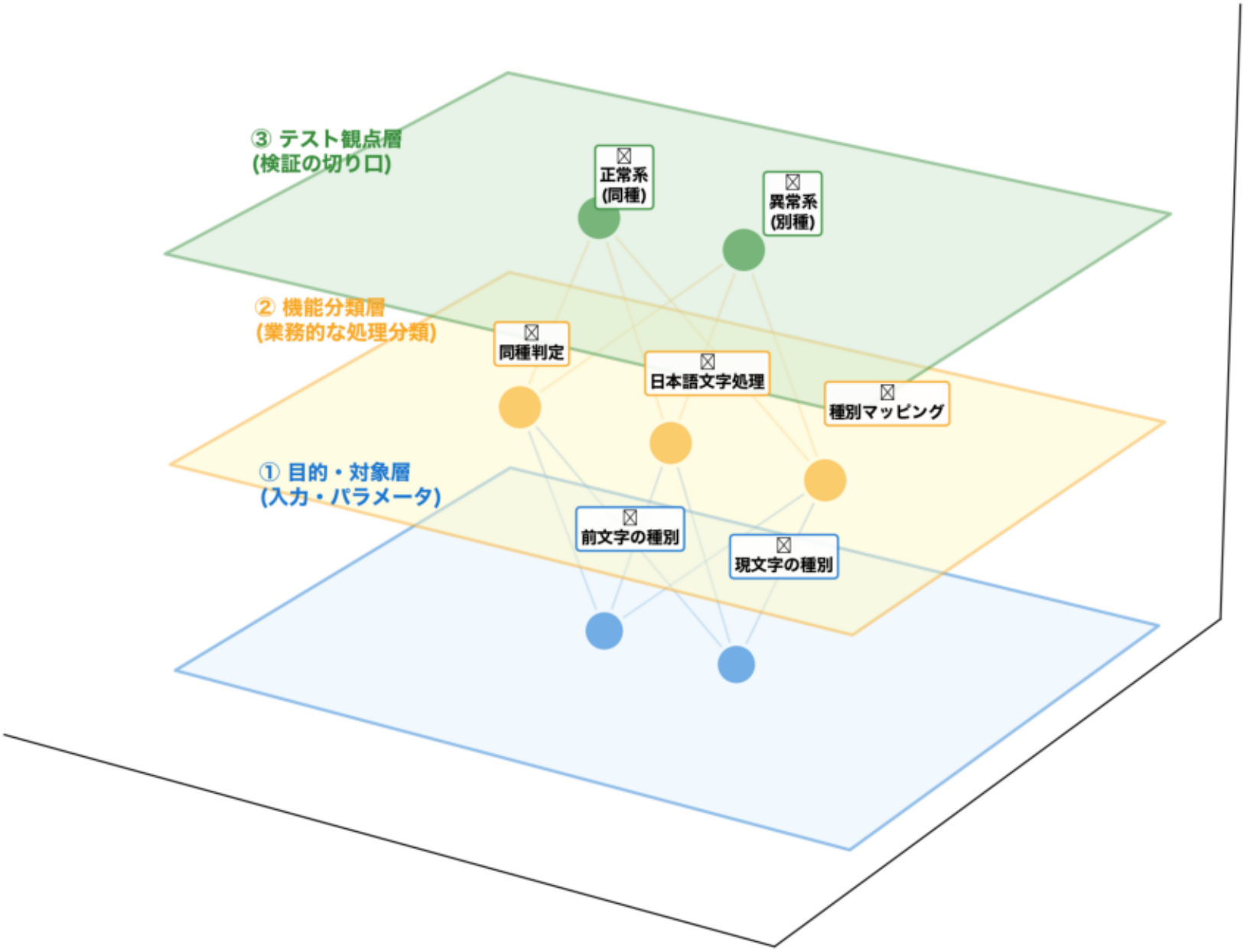
Dependency: sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars
— 依存関係 (手続き型代替) —

static ECharKind CWordParse::WhatKindOfTwoChars
独立関数 (外部依存なし)
要求数: 11

# 【機能分類ジオラマ】非エンジニア向け構造図

## 【sakura-editor】 static ECharKind CWordParse::WhatKindOfTwoChars

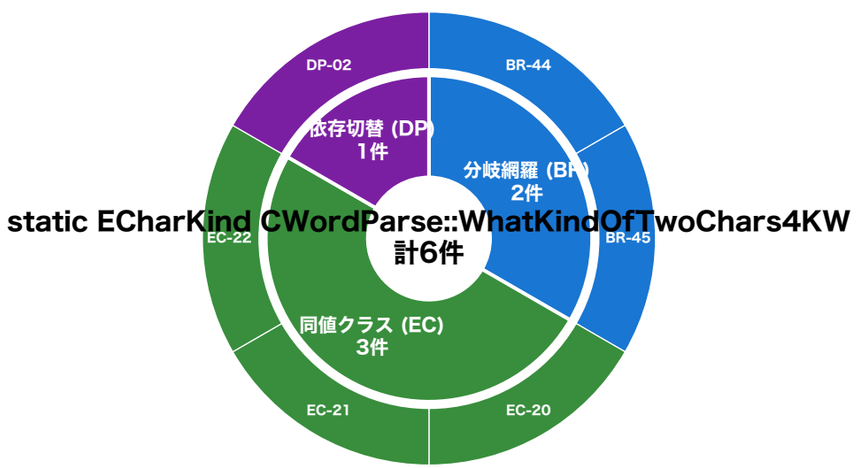
目的: 2文字の文字種関係を判定



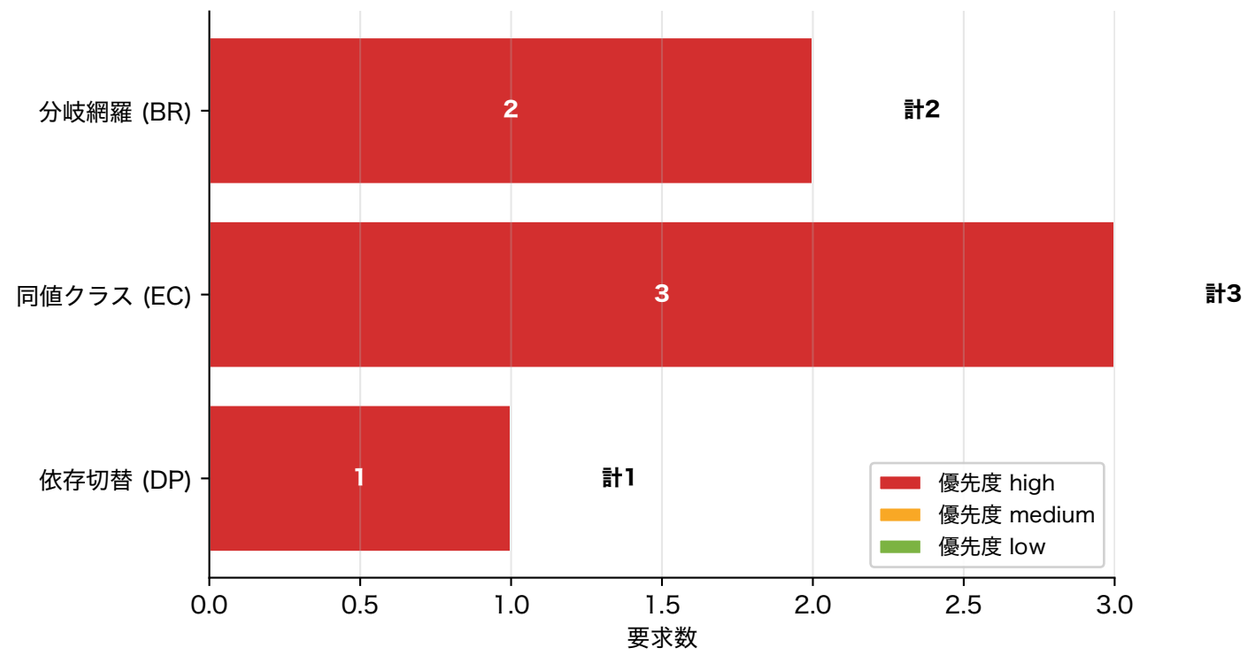

▼ 入力層 (何が入力か) → 機能分類層 (何を処理するか) → テスト観点層 (どう検証するか) ▼

# 【4種の可視化レイヤ】sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars4KW

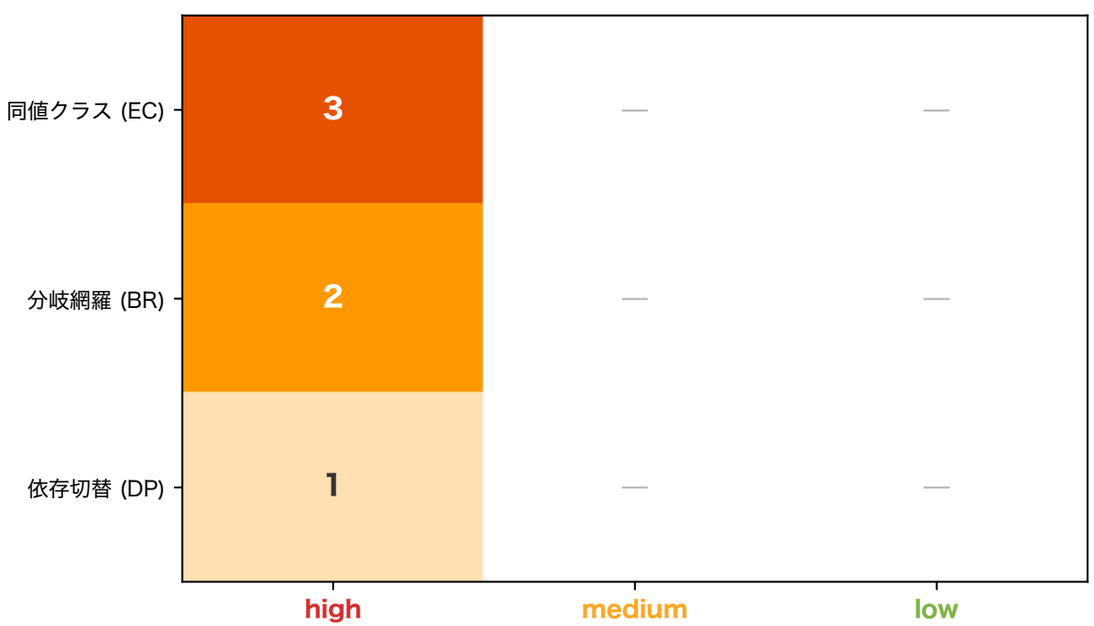
Sunburst: sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars4KW
— 要求の種別階層 —




Sankey: sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars4KW
— 種別×優先度の流量 —




Heatmap: sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars4KW
— 要求種別 × 優先度 (手続き型代替表示) —




計: high=6, medium=0, low=0

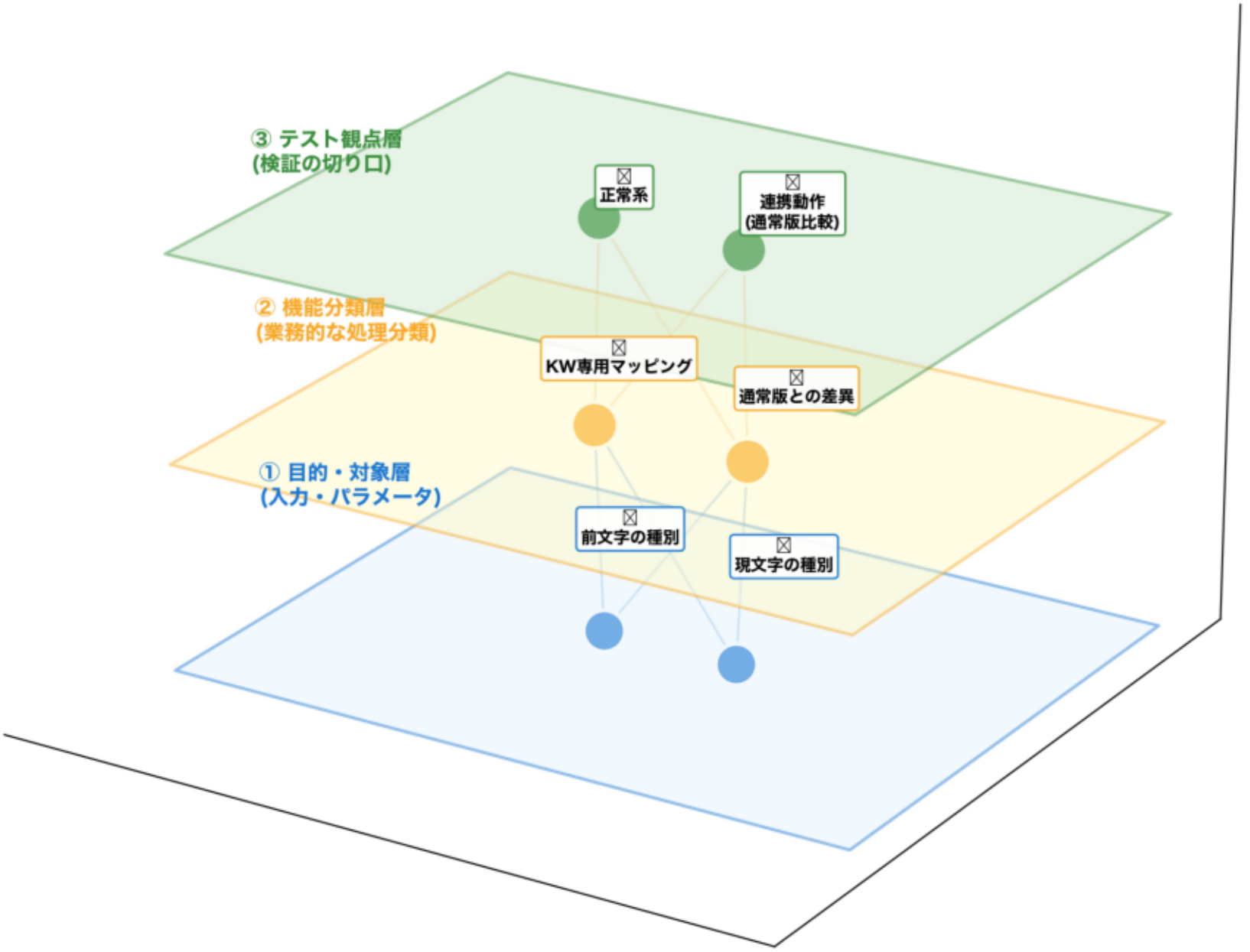
Chord: sakura-editor/sakura / static ECharKind CWordParse::WhatKindOfTwoChars4KW
— 依存関係 (手続き型代替) —

static ECharKind CWordParse::WhatKindOfTwoChars4KW
独立関数 (外部依存なし)
要求数: 6

# 【機能分類ジオラマ】非エンジニア向け構造図

## 【sakura-editor】 static ECharKind CWordParse::WhatKindOfTwoChars4KW

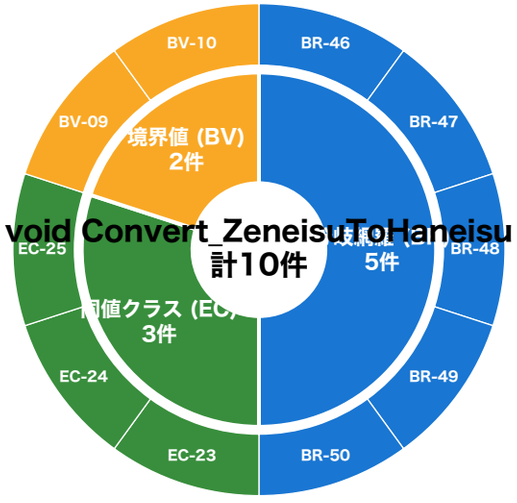
目的: キーワード用の文字種関係判定



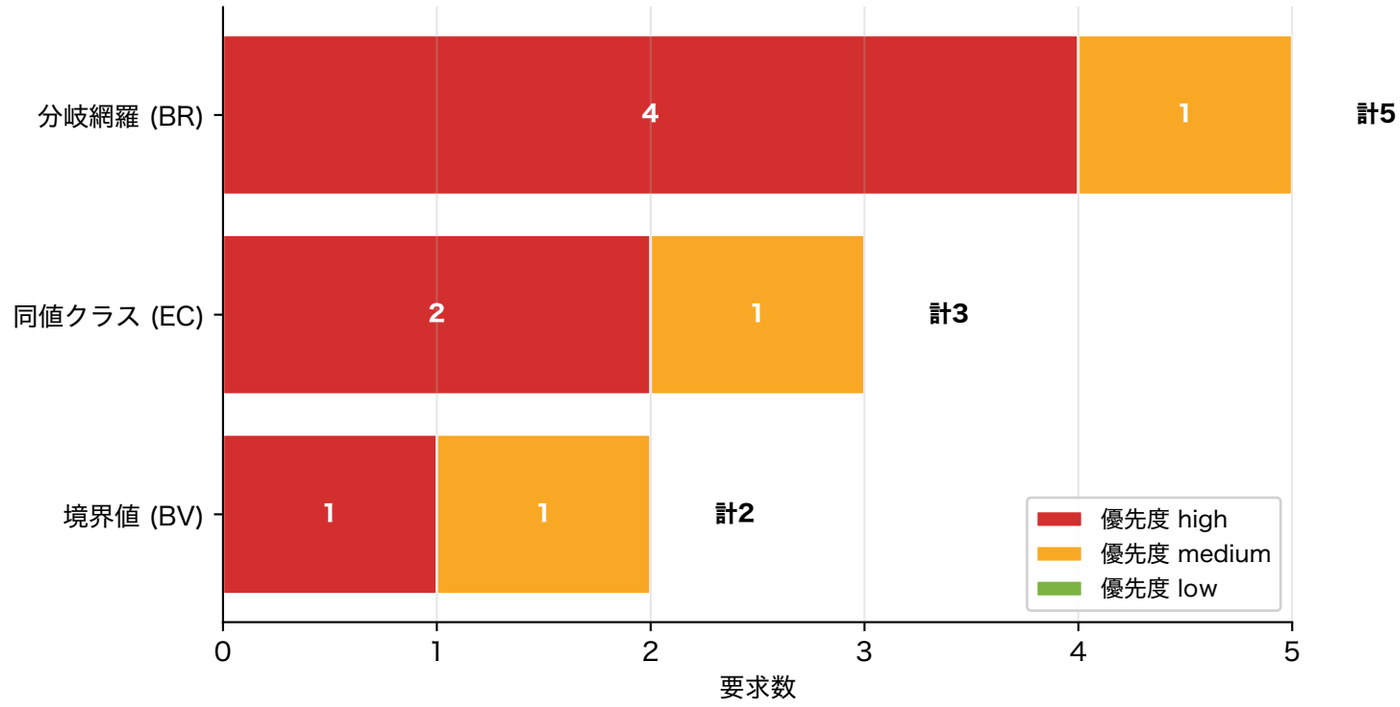

▼ 入力層 (何が入力か) → 機能分類層 (何を処理するか) → テスト観点層 (どう検証するか) ▼

# 【4種の可視化レイヤ】sakura-editor/sakura / void Convert_ZeneisuToHaneisu

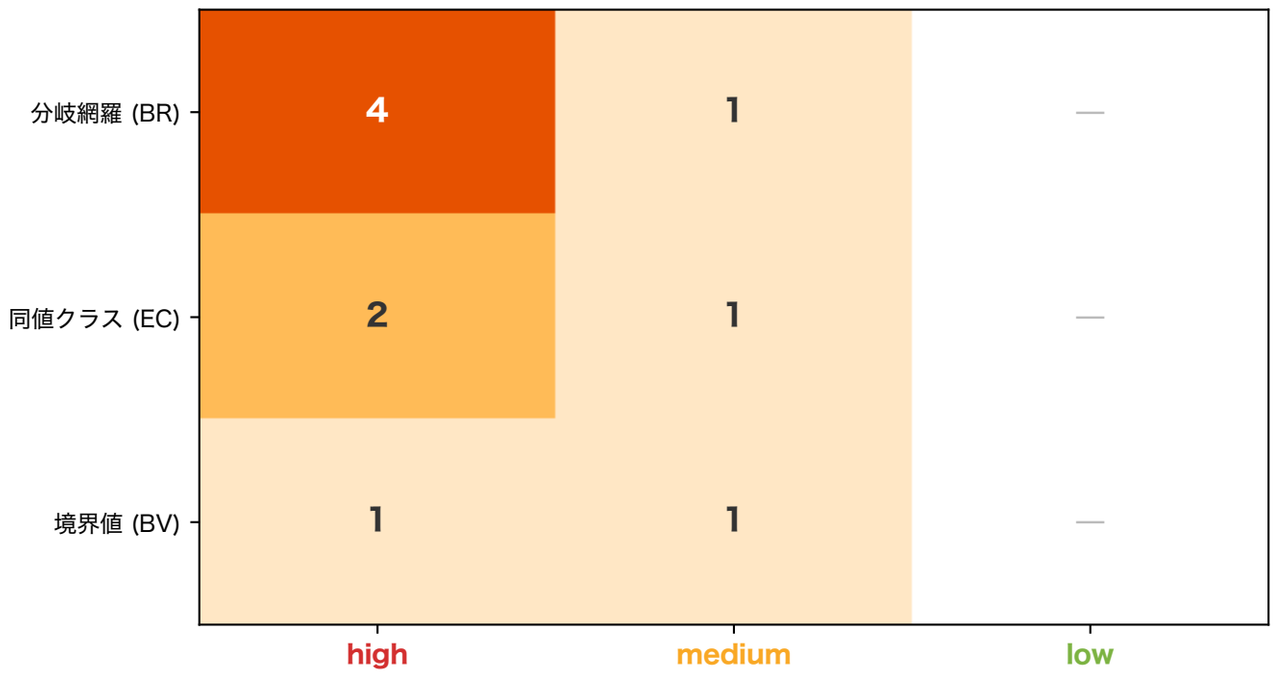
Sunburst: sakura-editor/sakura / void Convert_ZeneisuToHaneisu
— 要求の種別階層 —




Sankey: sakura-editor/sakura / void Convert_ZeneisuToHaneisu
— 種別×優先度の流量 —

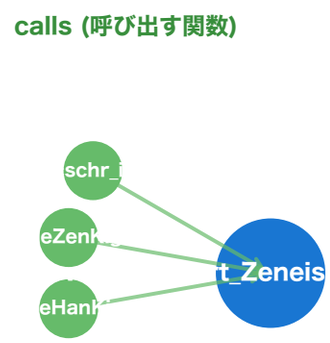



Heatmap: sakura-editor/sakura / void Convert_ZeneisuToHaneisu
— 要求種別 × 優先度 (手続き型代替表示) —




計: high=7, medium=3, low=0

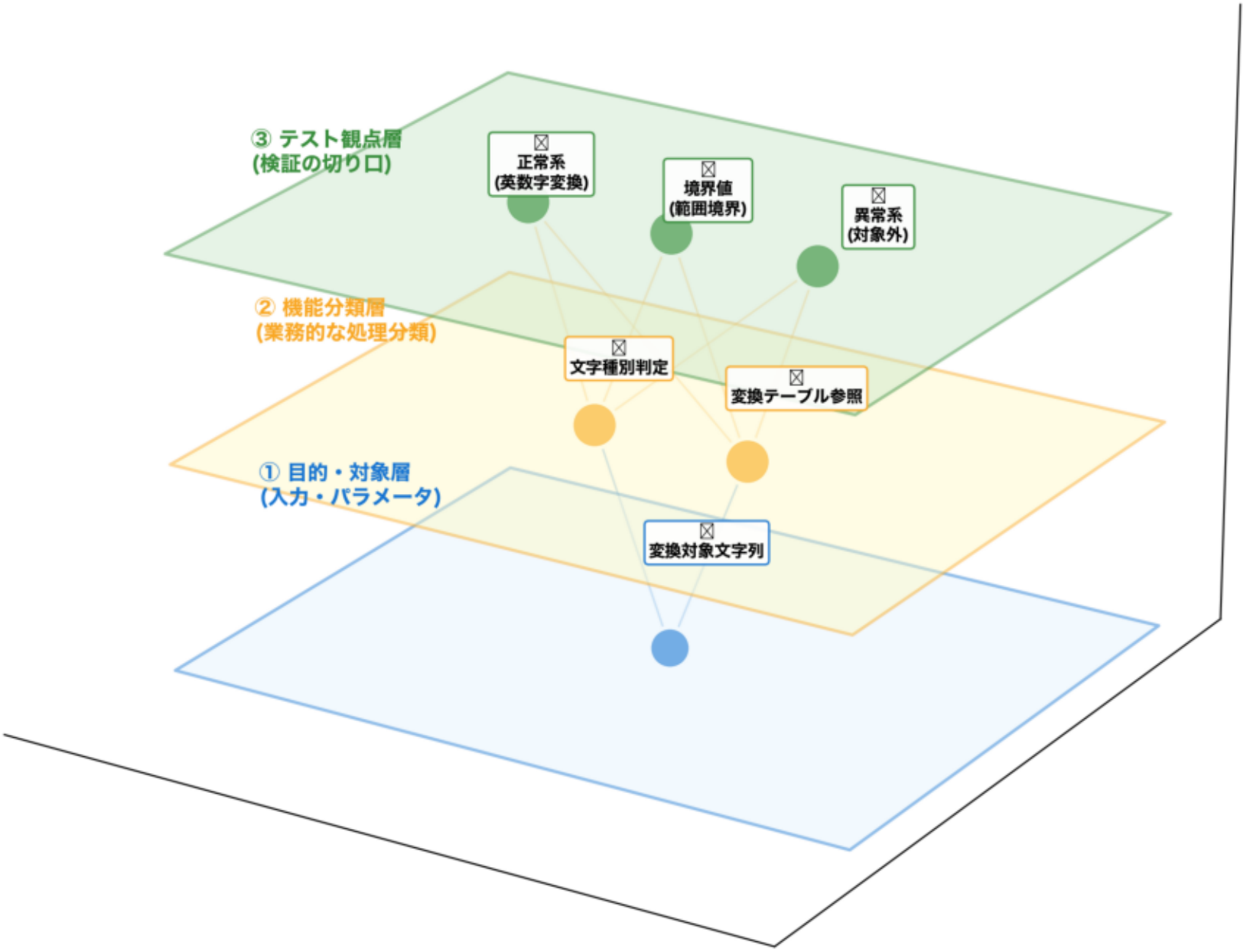
Chord: sakura-editor/sakura / void Convert_ZeneisuToHaneisu
— 依存関係マップ (手続き型代替) —



# 【機能分類ジオラマ】非エンジニア向け構造図

## 【sakura-editor】 void Convert_ZeneisuToHaneisu

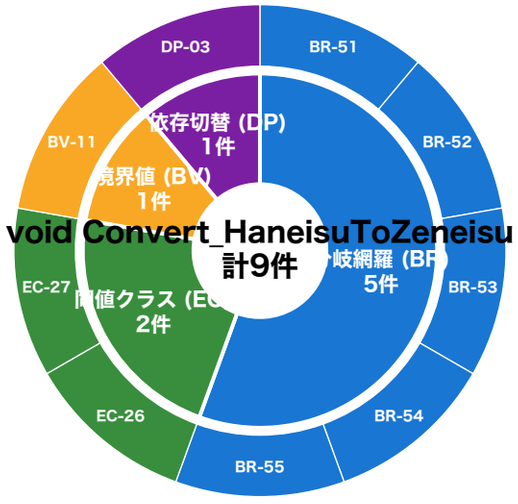
目的: 全角英数字を半角に変換



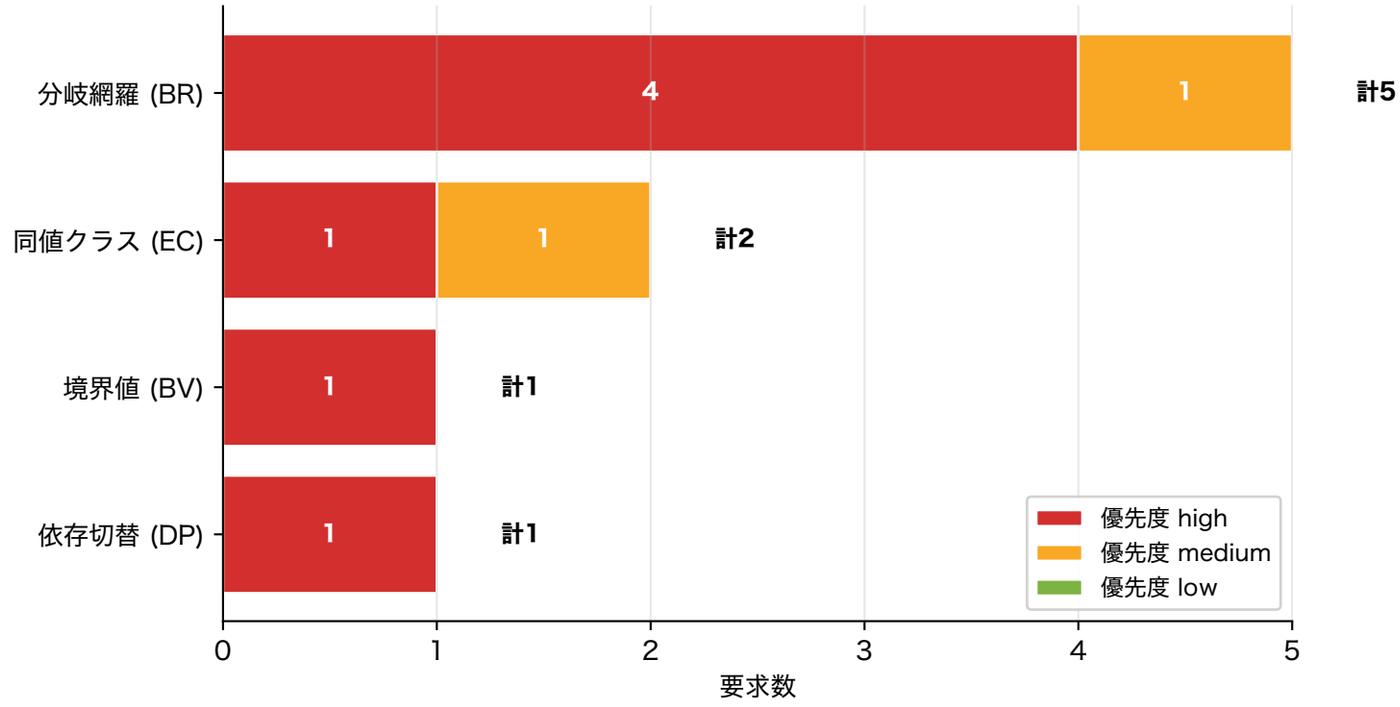

▼ 入力層 (何が入力か) → 機能分類層 (何を処理するか) → テスト観点層 (どう検証するか) ▼

# 【4種の可視化レイヤ】sakura-editor/sakura / void Convert_HaneisuToZeneisu

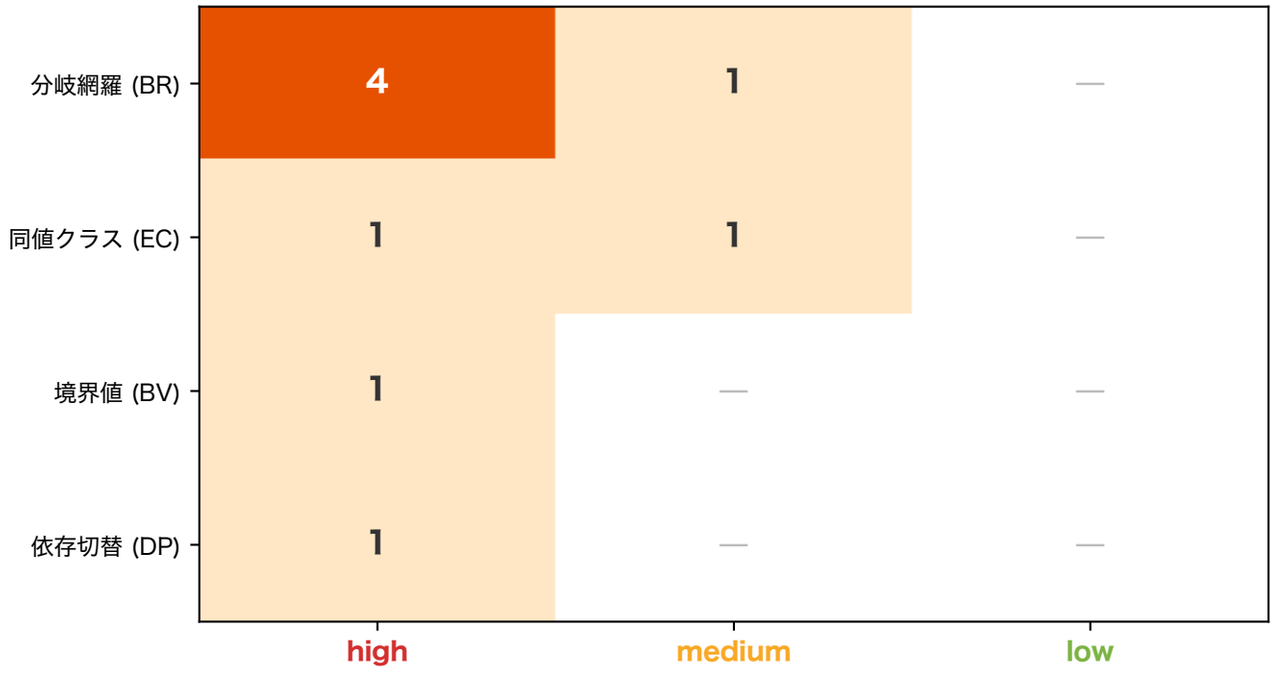
Sunburst: sakura-editor/sakura / void Convert_HaneisuToZeneisu
— 要求の種別階層 —




Sankey: sakura-editor/sakura / void Convert_HaneisuToZeneisu
— 種別×優先度の流量 —

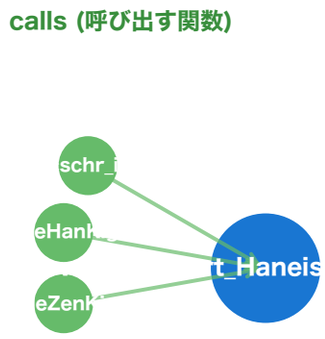



Heatmap: sakura-editor/sakura / void Convert_HaneisuToZeneisu
— 要求種別 × 優先度 (手続き型代替表示) —




計: high=7, medium=2, low=0

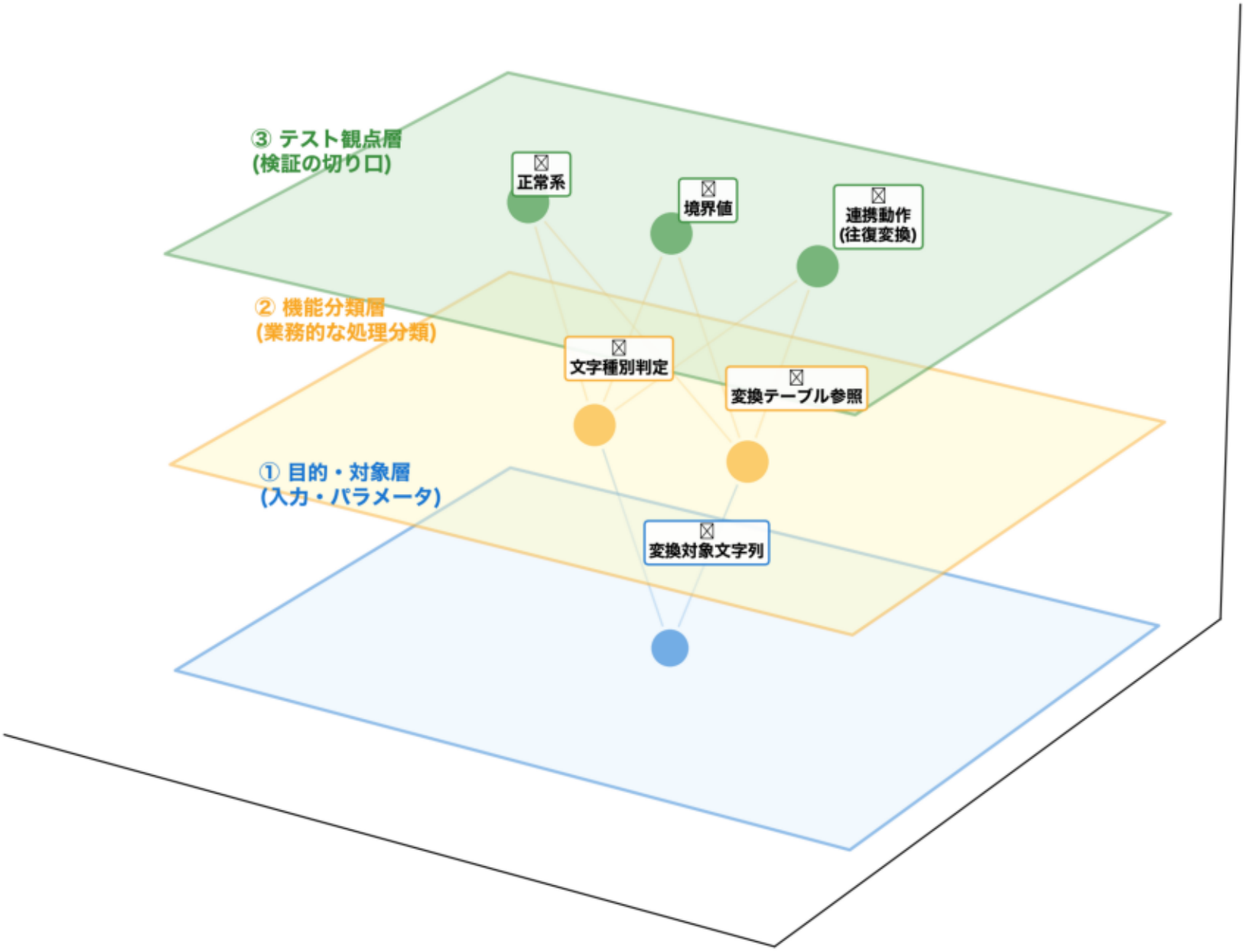
Chord: sakura-editor/sakura / void Convert_HaneisuToZeneisu
— 依存関係マップ (手続き型代替) —



# 【機能分類ジオラマ】非エンジニア向け構造図

## 【sakura-editor】 void Convert_HaneisuToZeneisu

目的: 半角英数字を全角に変換




▼ 入力層 (何が入力か) → 機能分類層 (何を処理するか) → テスト観点層 (どう検証するか) ▼